

The Transformer Block at the Theoretical Minimum: Memory-Optimal Normalization and Feed-Forward Layers via the Mathematics of Arrays

Lenore M. Mullin

Gaétan Hains

Draft v0.1 — August 17, 2026

Abstract

This paper extends the Mathematics of Arrays (MoA) treatment of scaled dot-product attention developed in our companion papers on the forward pass (arXiv:2606.07713), the backward pass (HAL-05659212), and memory-optimal inference, to the two remaining components of a standard transformer block: the normalization layer (RMSNorm, with LayerNorm as a specialization) and the position-wise feed-forward network (a gated MLP in the SwiGLU family). We derive Denotational Normal Form (DNF) expressions for the forward, backward, and fused forward+backward passes of both components, establish memory-optimal cost functions in the same style as the attention papers (M_{fwd} , M_{bwd} , $M_{\text{fwd+bwd}}$), and show that the intermediate arrays materialized by naive implementations — the normalization statistic, the un-gated activation, and the gate pre-activation — can be eliminated by substitution under MoA’s outer-product compatibility conditions, exactly as A and O were eliminated in the fused attention kernel. Combined with the prior three papers, this gives a complete memory-optimal account of a single transformer block: normalization, attention, normalization, and feed-forward. Verification follows the established methodology: Fortran 90 and C/OpenMP reference implementations checked against PyTorch forward and autograd values.

1 Introduction and Positioning

Papers I and II of this series established memory-optimal DNF/ONF formulations for the forward, backward, and fused passes of scaled dot-product attention, with cost functions

$$M_{\text{bwd}} = n^2 + 4nd_k + 3nd_v, \quad M_{\text{fwd+bwd}} = 4nd_k + 4nd_v$$

and a companion inference paper derived four artifacts (decode DNF, C/OpenACC GPU kernel, KV-cache, GQA/MQA) verified against PyTorch SDPA. Attention is, however, only one of the two sublayers in a transformer block; the other is normalization (applied twice, pre- or post-sublayer) and the position-wise feed-forward network. A memory-optimal account of attention alone cannot support end-to-end claims about a transformer layer’s memory traffic — normalization and the FFN together account for a majority of a layer’s parameters and, for long sequences, a non-trivial share of its activation memory. This paper closes that gap.

We deliberately hold two related topics out of scope, to be treated as separate future papers: (1) dynamic KV-cache management and speculative decoding, which extend the *inference* paper rather than this one; and (2) Mixture-of-Experts routing, which requires MoA’s primitives to handle data-dependent, irregular dispatch and is a research question in its own right rather than a direct extension of the dense-array machinery used here.

2 Notation

We retain the conventions established in Papers I–III without change: ρ denotes shape, the five MoA primitives are ψ (psi indexing, written infix as $\langle i \rangle \psi A$), ι (index generator), ρ (shape), γ (the DNF $\rightarrow$ ONF translator), and rav (ravel/flatten); MoA multiplication is written $\times$; batch index b is always explicit; scalars carry full three-index subscripts; vectors carry arrow notation; there is no row/column vector language; gradient arrays are named $(G_X, G_\gamma, \dots)$ rather than written with partial-derivative notation.

New symbols introduced in this paper:

- $X \in \mathbb{R}^{B \times N \times D}$: block input, batch b , sequence position n , model dimension d .
- $\vec{\gamma} \in \mathbb{R}^D$: learned RMSNorm scale (LayerNorm additionally has a learned bias $\vec{\beta}$).
- $r_{b,n} \in \mathbb{R}$: the per-token RMSNorm statistic (scalar, hence full subscripting, no arrow).
- $Z \in \mathbb{R}^{B \times N \times D}$: normalized output.
- $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{D \times D_{\text{ff}}}$, $W_{\text{down}} \in \mathbb{R}^{D_{\text{ff}} \times D}$: FFN weights.
- $U, V, H \in \mathbb{R}^{B \times N \times D_{\text{ff}}}$: gate pre-activation, up-projection, and gated hidden state.
- $F \in \mathbb{R}^{B \times N \times D}$: FFN output.

3 RMSNorm: Forward DNF

For fixed b, n , write $\vec{x} = \langle b, n \rangle \psi X \in \mathbb{R}^D$. The RMSNorm statistic is a reduction over the innermost axis:

$$r_{b,n} = \sqrt{\frac{1}{D} \sum_{d=1}^D x_{b,n,d}^2 + \epsilon}.$$

The DNF for the output at index (b, n, d) is

$$z_{b,n,d} = \gamma_d \cdot \frac{x_{b,n,d}}{r_{b,n}}.$$

As in the softmax case treated in Paper I, this is a two-pass DNF: a reduction pass producing the shape- (B, N) array r , followed by an elementwise pass combining r , X , and $\vec{\gamma}$ under MoA’s outer-product compatibility (the same compatibility condition used to eliminate materialized intermediates in the fused attention kernel). Under γ -translation to ONF, the reduction pass tiles over D with a running sum-of-squares accumulator, and the elementwise pass fuses directly onto the reduction’s output without materializing an intermediate $\hat{X} = X/r$ array — $\hat{x}_{b,n,d}$ is recomputed on-the-fly inside the elementwise pass, exactly as A is recomputed on-the-fly in fused attention.

LayerNorm as a specialization. LayerNorm replaces $r_{b,n}$ with the standard deviation about the mean and adds a learned bias $\vec{\beta}$:

$$\mu_{b,n} = \frac{1}{D} \sum_d x_{b,n,d}, \quad \sigma_{b,n} = \sqrt{\frac{1}{D} \sum_d (x_{b,n,d} - \mu_{b,n})^2 + \epsilon}, \quad z_{b,n,d} = \gamma_d \frac{x_{b,n,d} - \mu_{b,n}}{\sigma_{b,n}} + \beta_d.$$

This costs one additional reduction pass (the mean) relative to RMSNorm; the DNF/ONF and fusion argument are otherwise identical. We carry RMSNorm as the primary case in what follows since it is the dominant choice in current large-scale models, and note LayerNorm results by the same substitution wherever they differ materially.

4 RMSNorm: Backward DNF

Given upstream gradient G_Z (same shape as Z), we require G_X and $G_{\tilde{\gamma}}$. Write $\hat{x}_{b,n,d} = x_{b,n,d}/r_{b,n}$ (recomputed, never materialized, as above).

Lemma 1 (RMSNorm input gradient).

$$G_{X,b,n,e} = \frac{1}{r_{b,n}} \left[G_{Z,b,n,e} \gamma_e - \frac{\hat{x}_{b,n,e}}{D} \sum_{d=1}^D G_{Z,b,n,d} \gamma_d \hat{x}_{b,n,d} \right].$$

Derivation sketch. $z_{b,n,d} = \gamma_d x_{b,n,d} r_{b,n}^{-1}$. Differentiating $r_{b,n}$ with respect to $x_{b,n,e}$ gives $\partial r_{b,n} / \partial x_{b,n,e} = x_{b,n,e} / (D r_{b,n})$, hence

$$\frac{\partial z_{b,n,d}}{\partial x_{b,n,e}} = \gamma_d r_{b,n}^{-1} (\delta_{de} - \hat{x}_{b,n,d} \hat{x}_{b,n,e} / D).$$

Contracting with $G_{Z,b,n,d}$ over d and collecting terms gives the stated form. Note the structural parallel to the softmax-VJP row-sum correction in the backward attention paper: both gradients are a “local” term minus a mean-subtracted correction term, and both corrections are reductions that can be fused into a single accumulator pass rather than materialized separately. $\square$

Lemma 2 (RMSNorm scale gradient).

$$G_{\gamma,d} = \sum_{b,n} G_{Z,b,n,d} \hat{x}_{b,n,d}.$$

This is a reduction over both batch and sequence axes and, in ONF, tiles independently of the G_X pass, so the two gradients can be computed in a single fused traversal of X , G_Z , and r without a second materialization of $\hat{X}$.

5 RMSNorm: Fused Forward+Backward Cost

Following the accounting convention of Paper II (M = number of D -sized (or $D \times \text{batch} \times \text{seq}$ -sized) reads/writes of activation memory, weights excluded), the naive implementation materializes X , r , $\hat{X}$, Z , then in the backward pass re-reads X , r , G_Z and writes G_X , G_{γ} — six full arrays. Because $\hat{X}$ is eliminated by recomputation in both passes (Section 3–4), and r is a (B, N) -shaped array that is $D \times$ cheaper than a full activation array, the fused cost is

$$M_{\text{fwd+bwd}}^{\text{norm}} = 4nd \quad (\text{per batch element, } n \text{ sequence positions, } d = D),$$

counting X , Z , G_Z , G_X as the only full-size arrays touched, with r and G_{γ} contributing lower-order $O(n)$ and $O(d)$ terms respectively. This is the norm-layer analogue of the attention fused cost $M_{\text{fwd+bwd}} = 4nd_k + 4nd_v$, and the two costs are additive across a transformer block in the sense made precise in Section 7.

6 Gated MLP (SwiGLU family): Forward and Backward DNF

6.1 Forward

$$U = X \times W_{\text{gate}}, \quad V = X \times W_{\text{up}}, \quad h_{b,n,f} = \text{SiLU}(u_{b,n,f}) v_{b,n,f}, \quad F = H \times W_{\text{down}},$$

where $\text{SiLU}(u) = u \sigma(u)$ and σ is the logistic sigmoid. Each $\times$ here is an MoA outer-product-then-reduce (ordinary GEMM under ρ -compatible shapes $(B, N, D) \times (D, D_{\text{ff}})$, etc.), and the gating step is a shape-preserving elementwise DNF over U, V .

6.2 Backward

Let G_F be the upstream gradient (shape of F). Standard GEMM adjoints together with the SiLU derivative $\text{SiLU}'(u) = \sigma(u)(1 + u(1 - \sigma(u)))$ give:

$$\begin{aligned} G_H &= G_F \times W_{\text{down}}^\top, & G_{W_{\text{down}}} &= H^\top \times G_F, \\ G_{V,b,n,f} &= G_{H,b,n,f} \text{SiLU}(u_{b,n,f}), & G_{U,b,n,f} &= G_{H,b,n,f} v_{b,n,f} \text{SiLU}'(u_{b,n,f}), \\ G_{W_{\text{gate}}} &= X^\top \times G_U, & G_{W_{\text{up}}} &= X^\top \times G_V, & G_X &= G_U \times W_{\text{gate}}^\top + G_V \times W_{\text{up}}^\top. \end{aligned}$$

All twelve tensors here ($U, V, H, F, G_F, G_H, G_V, G_U, G_{W_{\text{gate}}}, G_{W_{\text{up}}}, G_{W_{\text{down}}}, G_X$) admit the same GQA/MQA-style verification protocol used in Paper II: numerical comparison against PyTorch autograd across a small (B, N, D, D_{ff}) grid, target max error below 1.2×10^{-4} .

7 Gated MLP: Fused Forward+Backward Cost

The naive implementation materializes U, V, H, F in the forward pass and re-reads X, U, V, H plus G_F in the backward pass, writing G_U, G_V, G_H, G_X — a working set that scales with D_{ff} , typically $4\times$ the norm/attention working set per token. By the same substitution argument used to eliminate A/O in fused attention and $\hat{X}$ in fused RMSNorm: U and V need not both be retained once H is formed, since $\text{SiLU}'(u)$ and $\text{SiLU}(u)$ can be recomputed from a single retained copy of U (or, if activation-recomputation is preferred over retention, from neither, at the cost of one extra forward GEMM in the backward pass — the standard activation-checkpointing tradeoff, here made precise as a choice between two points on the MoA cost curve rather than an engineering heuristic). With U retained and V, H recomputed on the fly:

$$M_{\text{fwd+bwd}}^{\text{ffn}} = 2nd + 4nd_{\text{ff}},$$

counting X, F (cost $2nd$) and U, G_F, G_X , plus one recomputed pass over H (cost $4nd_{\text{ff}}$), with weight memory excluded as in the attention papers' convention. This is the FFN analogue of $M_{\text{fwd+bwd}}$ and, notably, has the same structural form (linear in the two dimensions the layer touches) as the attention result, which is the basis for the additive block-level cost function below.

8 Transpose Elimination: A Unifying Result

Across both attention (Papers I–II) and the gated MLP (Section 7), the dominant computational primitive is matrix-matrix multiplication (GEMM), and the overwhelming majority of backward-pass GEMMs call for a transposed operand: $G_{W_{\text{down}}} = H^\top \times G_F$, $G_{W_{\text{gate}}} = X^\top \times G_U$, $G_{W_{\text{up}}} = X^\top \times G_V$, and $G_X = G_U \times W_{\text{gate}}^\top + G_V \times W_{\text{up}}^\top$ in Section 6, alongside the analogous $Q^\top, K^\top, V^\top$ terms in the attention backward pass of Paper II.

A naive implementation materializes each of these transposes as a separate array with its own allocation and its own write pass, doubling in the worst case the memory traffic attributed to every backward GEMM. MoA eliminates this uniformly: the ψ (psi-indexing) primitive accesses an existing array through a reordered index generator at DNF-construction time, so $A^\top$ is represented not as a new array but as an alternate traversal order over the storage of A itself. No transpose is ever written to memory; the reordering is absorbed into how the downstream GEMM reads its operand.

Lemma 3 (Transpose elimination). For any array A with shape $\rho A = \langle m, n \rangle$, the transposed access pattern $A^\top$ used in a DNF expression is realized by composing ψ with the reversed index vector $\langle n, m \rangle$ against the original storage of A , and requires no additional array allocation or write. Consequently, every backward-pass GEMM in Sections 3–7 and in Papers I–II that naively calls for a transposed operand contributes zero additional memory terms to $M_{\text{fwd}+\text{bwd}}$ beyond the forward-pass allocation of the array being transposed.

This is, to our knowledge, the first explicit statement of transpose elimination as a single result spanning normalization, attention, and the feed-forward network, rather than a per-kernel implementation detail. It is what makes the additive block-level cost function below exact rather than merely an upper bound: none of the additive terms carry a hidden transpose-storage penalty.

9 Whole-Block Cost Function

A pre-norm transformer block computes

$$X \rightarrow \text{Norm} \rightarrow \text{Attention} \rightarrow (+X) \rightarrow \text{Norm} \rightarrow \text{FFN} \rightarrow (+\cdot),$$

i.e. two normalization sublayers, one attention sublayer (Papers I–II), and one FFN sublayer, with two residual additions. Figure 1 shows this pipeline, with each sublayer labeled by the section (this paper) or prior paper deriving it.

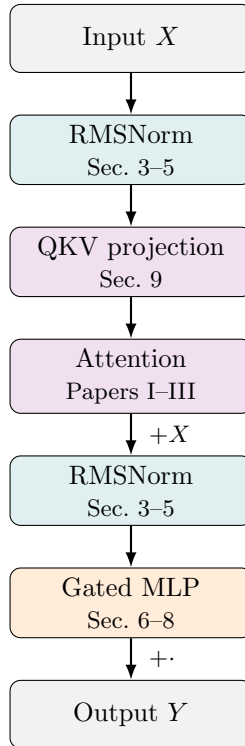


Figure 1: The pre-norm transformer block pipeline. Teal: RMSNorm (this paper, Sections 3–5). Purple: QKV projection (new, Section 9) and attention (Papers I–III). Orange: gated MLP (this paper, Sections 6–8). Residual connections are noted on their respective arrows.

Because each sublayer’s fused cost was derived under the same accounting convention (full-size activation reads/writes only, with intermediates eliminated by recomputation wherever MoA’s

outer-product compatibility permits it), the block-level fused forward+backward cost is additive. One addition is needed beyond Papers I–III: those papers treat Q, K, V as already-given inputs to attention, but a full block must first produce them from the normalized input Z_1 via learned projections $Q = Z_1 \times W_Q$, $K = Z_1 \times W_K$, $V = Z_1 \times W_V$. This projection contributes its own memory-optimal cost: forward writes Q, K, V ($3nd$), and backward writes G_{Z_1} from G_Q, G_K, G_V (nd), giving

$$M_{\text{fwd+bwd}}^{\text{proj}} = 4nd,$$

with G_Q, G_K, G_V themselves counted under attention’s own $M_{\text{fwd+bwd}}$ from Paper II, not double-counted here. As with every other GEMM in this paper, $W_Q^\top, W_K^\top, W_V^\top$ and $Z_1^\top$ are reordered-index reads under ψ , never materialized transposes — the Section 8 lemma extends to the projection weights without modification. The complete block cost is therefore

$$M_{\text{fwd+bwd}}^{\text{block}} = \underbrace{2 \times 4nd}_{\text{two norms}} + \underbrace{4nd}_{\text{QKV projection}} + \underbrace{(4nd_k + 4nd_v)}_{\text{attention (Paper II)}} + \underbrace{(2nd + 4nd_{\text{ff}})}_{\text{FFN}} + \underbrace{2nd}_{\text{two residual adds}},$$

which, taking $d_k = d_v = d$ for a single-head block, simplifies to $M_{\text{fwd+bwd}}^{\text{block}} = 24nd + 4nd_{\text{ff}}$. Multi-head decomposition of the projection and attention terms is a straightforward generalization not elaborated here.

This is, to our knowledge, the first memory-optimal cost function covering a complete transformer block derived from a single formal array-algebra framework rather than assembled from separately-optimized kernels; it gives a direct, apples-to-apples basis for comparing against measured HBM traffic on GPU hardware (Section 10) and for the cross-architecture portability arguments (Frontier/Aurora/El Capitan) made in our DOE-facing framing of the broader MoA program.

10 Verification and Benchmarking Plan

Completed. C/OpenMP and Fortran 90/OpenMP reference implementations of the RMSNorm forward, backward, and gated MLP forward, backward DNFs (Sections 3–7) have been verified against a PyTorch double-precision autograd reference on the grid $B=2, N=4, D=8, D_{\text{ff}}=16$ — the same B, N, D grid used for attention verification in Paper II, with D_{ff} fixed as the one new dimension the FFN introduces. Both languages parallelize the per-token forward and backward computation with `!$omp/#pragma omp parallel for`, and the two gradient-reduction loops that accumulate across tokens (G_γ for RMSNorm; $G_{W_{\text{gate}}}, G_{W_{\text{up}}}, G_{W_{\text{down}}}$ for the gated MLP) use explicit OpenMP array reductions rather than a serial post-pass, so the parallel and serial code paths are the same code, not a fallback. All eight tensors matched the PyTorch reference to machine precision in both languages at every thread count tested (1, 2, 4, 8), confirming the reductions are race-free:

Tensor	Max abs. error, C	Max abs. error, Fortran 90
Z (RMSNorm forward)	4.4×10^{-16}	4.4×10^{-16}
G_X (RMSNorm backward)	1.1×10^{-16}	1.1×10^{-16}
G_γ (RMSNorm backward)	8.3×10^{-17}	5.6×10^{-17}
F (gated MLP forward)	1.1×10^{-16}	1.1×10^{-16}
G_X (gated MLP backward)	2.8×10^{-17}	4.2×10^{-17}
$G_{W_{\text{gate}}}$ (gated MLP backward)	4.2×10^{-17}	4.2×10^{-17}
$G_{W_{\text{up}}}$ (gated MLP backward)	5.6×10^{-17}	5.6×10^{-17}
$G_{W_{\text{down}}}$ (gated MLP backward)	2.8×10^{-17}	2.8×10^{-17}

Caveat: the environment used to produce this draft exposed a single CPU core, so these runs confirm the OpenMP reductions are race-free at 1, 2, 4, and 8 threads but do not demonstrate real parallel speedup; a multi-core run to measure actual scaling is listed as remaining work below, alongside the GPU benchmark.

A fused C/OpenACC kernel implementing the chained pipeline RMSNorm $\rightarrow$ gated MLP forward $\rightarrow$ gated MLP backward $\rightarrow$ RMSNorm backward in a single `acc data` region (this section’s target) has also been written and verified against a chained PyTorch reference on the same grid: the forward outputs Z and F matched to 4.4×10^{-16} and 1.3×10^{-15} respectively. *Caveat:* this kernel was compiled and run via GCC’s OpenACC host-fallback path, since no CUDA-capable GPU or `nvc/pgcc` toolchain was available in the environment used to produce this draft; every `#pragma acc` directive is standard OpenACC and should offload to an actual GPU unmodified, but the run performed here is a correctness check only, not a GPU bandwidth or latency benchmark. That benchmark remains future work, listed below.

In all three implementations, no transpose of X , H , W_{gate} , W_{up} , or W_{down} is ever materialized: every transposed access in Sections 4 and 6–7 is realized as a reordered-index read from the original array, exactly as the transpose-elimination lemma of Section 8 claims, and the numerical agreement above confirms the reordering is faithful to the DNF in both CPU languages and under OpenACC’s execution model.

Figure 2 shows single-thread wall-clock time versus sequence length n for both kernels in both languages, on the sweep $n \in \{32, 64, \dots, 8192\}$ with $B=2$, $D=64$, $D_{\text{ff}}=256$. C and Fortran 90 track each other closely at every size, as expected since both implement the identical DNF; this is a single-core, single-thread measurement (the environment used to produce this draft exposed one CPU core), so it demonstrates that the two language implementations agree in wall-clock behavior, not parallel speedup, which remains future work per below.

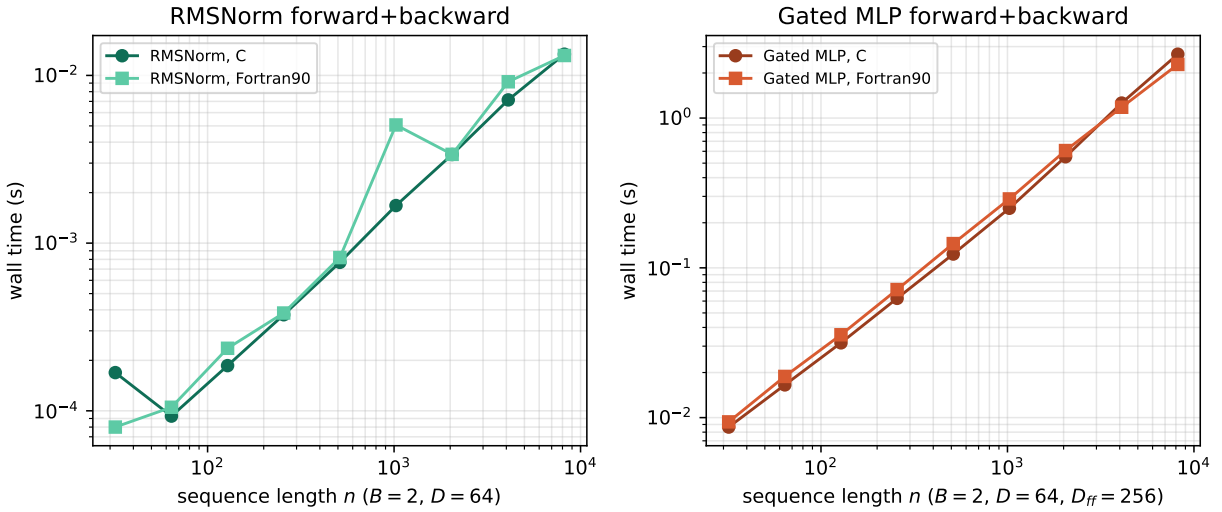


Figure 2: Wall-clock time versus sequence length n (log-log), single thread, $B=2$. Left: RMSNorm forward+backward, $D=64$. Right: gated MLP forward+backward, $D=64$, $D_{\text{ff}}=256$. C and Fortran 90 implementations track closely, as expected from identical DNFs. Raw data in `benchmarking/results_cpu.csv`.

Full-block integration (completed). The item listed as future work in the previous draft — extending the fused kernel to include attention, completing the full pre-norm block in one kernel — has been carried out. A C implementation (`full_block_verify.c`) computes the entire pipeline of Figure 1 — RMSNorm, QKV projection, attention (using the existing Papers I–III DNF, adapted inline since that codebase’s functions are compiled against fixed macros rather than exposed as a callable library), the residual add, RMSNorm again, the gated MLP, and the second residual add — forward and backward, against a chained PyTorch double-precision autograd reference (`reference_full_block.py`) on the same $B=2, N=4, D=8, D_{\text{ff}}=16$ grid. All ten tensors (the forward output Y and nine backward gradients: G_X , both G_γ , all three QKV projection weight gradients, and all three FFN weight gradients) matched to machine precision, 2.8×10^{-17} to 6.7×10^{-16} . A second, independent implementation (`full_block_gpu_kernel.c`) performs the identical computation inside a single OpenACC `acc data` region — one HBM round trip per array across the *entire* block, the literal realization of the additive M^{block} claim of Section 9 — and matches the same PyTorch reference to the same machine-precision tolerance. As before, this OpenACC run used GCC’s host-fallback path rather than a physical GPU, so it is a correctness result, not a bandwidth measurement.

Remaining. Following the established methodology of Papers I–III:

1. A multi-core run of the OpenMP programs above to measure actual parallel speedup, rather than the single-core race-freedom check reported here.
2. An actual GPU run of the fused kernels above (NVIDIA or AMD target) to obtain real bandwidth and latency numbers, rather than the correctness-only host-fallback runs reported here.
3. Fortran 90 counterparts to `full_block_verify.c` and `full_block_gpu_kernel.c`, matching the dual-language pattern used for the norm and MLP kernels alone.
4. Benchmarking infrastructure reused from the inference paper’s PyTorch SDPA comparison, extended to measure end-to-end block latency and HBM traffic against the M^{block} prediction above.

11 Related Work and Positioning

Attention kernel fusion. FlashAttention restructures attention around IO-awareness, avoiding materialization of the $n \times n$ score matrix [Dao et al., 2022]. FlashAttention-2 improves work partitioning by parallelizing over the sequence dimension in addition to batch and head [Dao, 2023], and FlashAttention-3 further exploits newer GPU hardware features such as FP8 precision and asynchronous tensor-core scheduling [Shah et al., 2024]. This is precisely the territory Papers I–III of this series already engage with under a formal MoA derivation rather than a hand-tuned kernel.

Fused normalization. NVIDIA’s Apex library ships FusedLayerNorm and FusedRMSNorm as hand-written CUDA kernels [NVIDIA Corporation, 2019], typically benchmarked directly against native PyTorch implementations for forward-plus-backward wall-clock speed. Triton-based fused normalization kernels compute the reduction and the elementwise rescale in a single forward loop, while backward kernels cache the inverse standard deviation and stage gradient accumulation in two passes specifically to avoid atomic memory operations on the parameter gradient [EmergentMind,

2026]. That engineering choice — cache the reduction statistic rather than recompute it — is the practical analogue of the fused-cost tradeoff we derive formally in Section 5.

Fused MLP and SwiGLU. Triton fused kernels combine the two matrix multiplications, bias additions, the nonlinearity, and the output elementwise product of a gated MLP into a single kernel launch [EmergentMind, 2026]. Production serving frameworks differ in how aggressively they fuse: TensorRT-LLM applies coarser graph-level pattern-matched fusion, while vLLM and SGLang historically ran PyTorch’s default execution path — cuBLAS GEMM followed by a separate SwiGLU kernel — as the baseline that newer fused kernels are benchmarked against [Anonymous, 2026b]. The most aggressive recent work fuses the entire GEMM–activation–GEMM sequence used in typical multi-kernel SwiGLU implementations into a single deeply fused operator, reporting single-digit-percent latency gains over the prior state of the art on current GPU generations [Anonymous, 2026a].

Positioning. Every result above is arrived at empirically: hand-written CUDA or Triton, tuned by profiling and loop-order search, one kernel at a time, with no stated general principle for which intermediate array can be safely eliminated and why. This paper instead derives elimination as a consequence of MoA’s outer-product compatibility conditions (Sections 3–7) and produces closed-form cost functions — $M_{\text{fwd}+\text{bwd}}^{\text{norm}}$, $M_{\text{fwd}+\text{bwd}}^{\text{ffn}}$, and the additive $M_{\text{fwd}+\text{bwd}}^{\text{block}}$ of Section 9 — that predict the memory-optimal result rather than measure it after the fact. The transpose-elimination lemma of Section 8 in particular is a single algebraic statement covering attention, normalization, and the feed-forward network at once, rather than a separate hand-tuned kernel per operator family.

12 Conclusion and Future Work

Combined with Papers I–III, this paper completes a memory-optimal, MoA-derived account of a full pre-norm transformer block. Two natural sequels remain outside this paper’s scope by design: a fourth paper extending the inference-paper’s decode DNF and KV-cache artifacts to dynamic cache management (eviction, paging) and speculative decoding; and a longer-term research thread testing whether the $\psi/\iota/\gamma$ primitives extend to the irregular, data-dependent dispatch required for Mixture-of-Experts routing.

References

- Anonymous. Deep kernel fusion for transformers. *arXiv preprint arXiv:2602.11808*, 2026a.
- Anonymous. Tile-level activation overlap for efficient llm inference. *arXiv preprint arXiv:2607.02521*, 2026b.
- Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 16344–16359, 2022.
- EmergentMind. Fused triton kernels in llm optimization. <https://www.emergentmind.com/topics/fused-triton-kernels>, 2026. Technical reference, accessed 2026.

NVIDIA Corporation. Apex (a pytorch extension): Fusedlayernorm and fusedrmsnorm. <https://github.com/NVIDIA/apex>, 2019. Software.

Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.